

NEAR DEV

SPUTNIKDAO2 SMART CONTRACT AUDIT

Q2-2025

COMPREHENSIVE PUBLIC REPORT

Prepared by:
Valhalla Security Consulting LLC
April 29, 2025





DISTRIBUTION (Public Version)

This report is intended for general informational use and provides an overview of the security testing performed by Valhalla Security Consulting LLC (Valhalla Security). All proprietary or sensitive details from the original assessment have been removed to ensure this document can be shared publicly.

The testing approach followed industry-standard penetration testing practices, adapted by Valhalla Security for this engagement. This public version may be freely distributed, referenced, or included in external communications as needed.

CONFIDENTIALITY (Public Version)

This public report has been prepared to summarize the high-level findings of the security assessment in a manner suitable for external audiences. All confidential, sensitive, or identifying information has been excluded.

Valhalla Security maintains strict confidentiality regarding all client security data. More detailed technical results, artifacts, or sensitive findings are contained only in the private version of this report and are shared exclusively with authorized client stakeholders.

ASSESSMENT TEAM

The following Valhalla Security personnel are involved in this engagement. Contact the appropriate personnel on this team to discuss the contents of this document or the work performed during this engagement.

PRIMARY SECURITY CONSULTANT

Ron Stoner
Sr Code Auditor

ADDITIONAL SECURITY CONSULTANTS

Sean Martin
CEO – Valhalla Security

Alex Bass
CTO – Valhalla Security



Table of Contents

Executive Summary.....	3
Engagement Scope of Work.....	5
Strengths and Weaknesses.....	5
Methodology.....	7
Identified Risks.....	8
<i>Potential Funds Loss in Upgrades.....</i>	<i>8</i>
<i>Staking Contract Immutability.....</i>	<i>10</i>
<i>Missing Overflow/Underflow Checks.....</i>	<i>11</i>
<i>Insufficient Validation in act_proposal.....</i>	<i>12</i>
<i>Improper Validation on Token Transfers.....</i>	<i>13</i>
<i>Insufficient Access Controls – delegate/undelegate.....</i>	<i>14</i>
<i>Possible Re-entrancy.....</i>	<i>15</i>
<i>Time-based Vulnerabilities.....</i>	<i>16</i>
<i>Unbounded Iterations.....</i>	<i>17</i>
<i>Lack of Duplicate Proposal Prevention.....</i>	<i>18</i>
<i>Inconsistent Error Handling.....</i>	<i>19</i>



Executive Summary

NEAR DEV (NEAR) enlisted the services of Valhalla Security to perform a time-boxed code audit assessment of the organization's Sputnik DAO contracts in:

<https://github.com/near-daos/sputnik-dao-contract/tree/main/sputnikdao2/src>

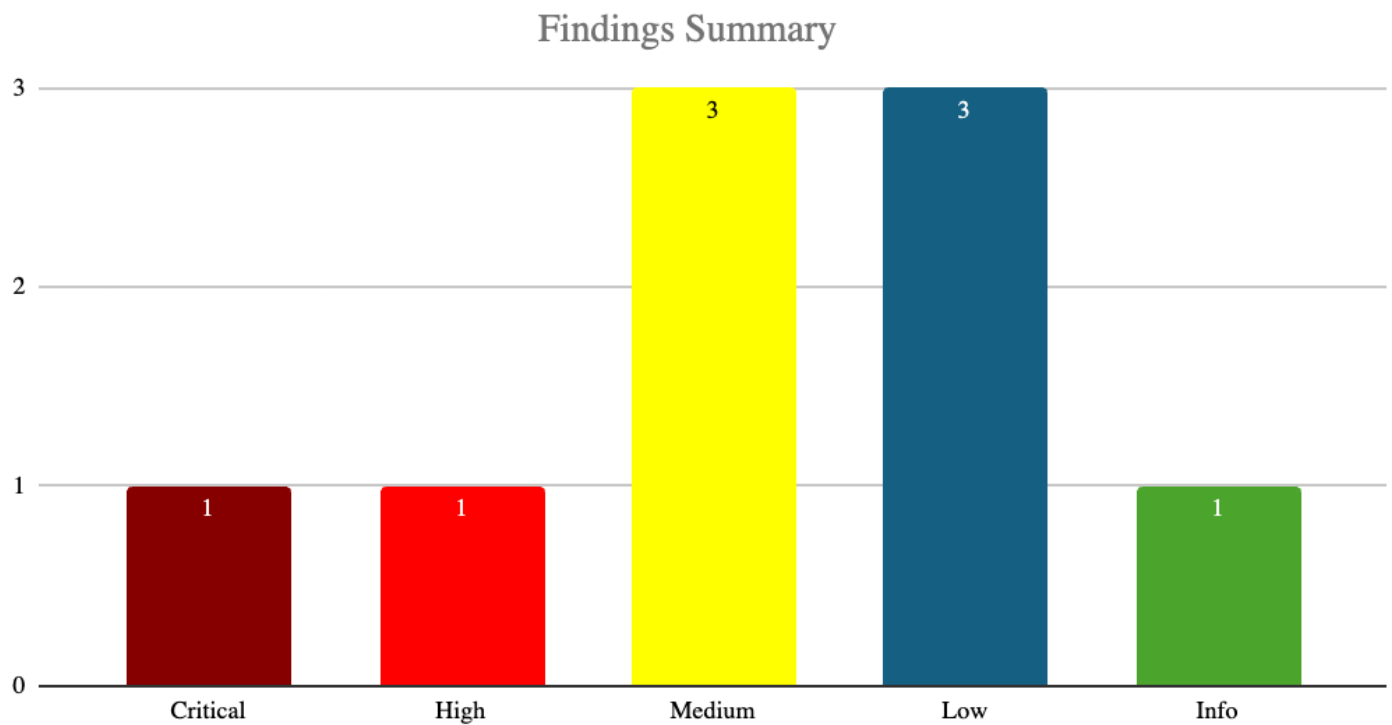
Commit Hash: 278adc713e795f95a6da4d3007c7e03e8120f153

A phased approach to the code audit included information gathering, manual and automated testing, source code review, and adherence to best practices. The areas of audit focus were:

- Access Control
- Arithmetic Operations
- Fund Handling
- Input Validation
- Upgrade Mechanisms

Valhalla Security has identified several threats that present risks of compromise and upstream risks. While the threats are rated high for severity, the likelihood is low due to most attacks requiring key compromise or malicious approvals via governance. Consultants noted the foundational level calls were sound and implemented properly following best practices.

To aid in the understanding of where threats lie within the organization, a breakout of threats based on criticality of the assessment is displayed below.





Opportunities for improvements are identified below:

Critical Risk	
Potential Funds Loss in Upgrades - FIXED	
High Risk	
Staking Contract Immutability - ACKNOWLEDGED	
Medium Risk	
Missing Overflow/Underflow Checks - ACKNOWLEDGED	
Improper Validation on Token Transfers - ACKNOWLEDGED	
Insufficient Access Controls – delegate/undelegate - ACKNOWLEDGED	
Low Risk	
Time-based Vulnerabilities - ACKNOWLEDGED	
Unbounded Iterations - ACKNOWLEDGED	
Lack of Duplicate Proposal Prevention - ACKNOWLEDGED	
Informational	
Inconsistent Error Handling - ACKNOWLEDGED	



Engagement Scope of Work

The following gives a summary of the scope of work, phases, and top identified strengths and weaknesses during the assessment.

SCOPE OF THE ENGAGEMENT

- **Manual and Automated Testing** — Valhalla Security focused on the security of DAO contract in scope for the assessment. During this phase, consultants used a comprehensive set of tools, custom scripts, and manual techniques to thoroughly identify possible threats to the application. All identified threats were tested and validated to evaluate the depth of compromise and risk of exposure.
- **Source Code Review** — Valhalla Security examined DAO contract source code for security vulnerabilities and misconfigurations. Consultants conducted both manual and automated reviews of code supplied and then verified identified issues using the running applications when necessary.

Strengths and Weaknesses

The lists below outline the most significant strengths and weaknesses for each phase of testing. Each phase receives a summary threat exposure rating, then a final total summary threat exposure rating is shown.

IDENTIFIED STRENGTHS

- **Foundational Level Calls** — No issues with foundational level calls were identified during testing. They are implemented properly following best practices.

IDENTIFIED WEAKNESSES

- **Upstream Risks:**
 - Significant issues exist in the application's authorization and authentication." would be more
 - The system has critical flaws in its trust hierarchy where dependent components (like the factory contract or staking contract) can manipulate the core contract's state without sufficient verification, creating vulnerability chains that propagate from trusted components to the main application.
- **Risk of Compromise:**
 - The application contains fundamental security weaknesses in validation and verification processes that could allow malicious actors to gain unauthorized access to protected functions or resources, potentially leading to system takeover or asset theft.

This report details a snapshot of SputnikDAO2 security as of April 2025. The overall state of security could have improved, deteriorated, or remained unchanged since the conclusion of this assessment.



OVERALL THREAT EXPOSURE



The overall threat exposure for the assessment was ranked at a Low level of risk. This ranking derives from the severity rankings, as well as environmental and mitigating factors identified during the assessment.



Methodology

Time-Boxed Code Audit

This phase of the security assessment focused on the security of application(s) in scope for the assessment. During this phase, Valhalla Security consultants used a comprehensive set of tools, custom scripts, and manual techniques to thoroughly identify possible threats to the application. During the assessment, all identified threats were tested and validated to evaluate the depth of compromise and risk of exposure.

Code Audit Scope

As determined prior to the engagement by client personnel, the table below lists the targeted scope of this phase of the assessment:

SCOPE
SputnikDAO2 contracts source code
Unit testing and fuzzing

Tools and References

To fully assess the targets listed above, Valhalla Security utilized the following tools and techniques:

TOOLS AND REFERENCES USED
near-cli
cargo-clippy (linter)
cargo-fuzz (fuzzer)
https://github.com/gagdiez/reentrancy_attack
https://medium.com/@dehvcurtis/top-rust-smart-contract-vulnerabilities-a-deep-dive-with-examples-eb36a84c80
Ob
https://github.com/slowmist/solana-smart-contract-security-best-practices

Tasks Performed

To fully assess the targets listed above, Valhalla Security performed the following tasks:

TASKS PERFORMED
Application Information Gathering
Source Code Review
Automated Testing and Fuzzing
Manual Crawl and Testing
Validation Testing of Identified Threats

The remainder of this section of the deliverable details threats that were discovered and their relative threat severity as a result of the testing that took place.



Identified Risks

Potential Funds Loss in Upgrades



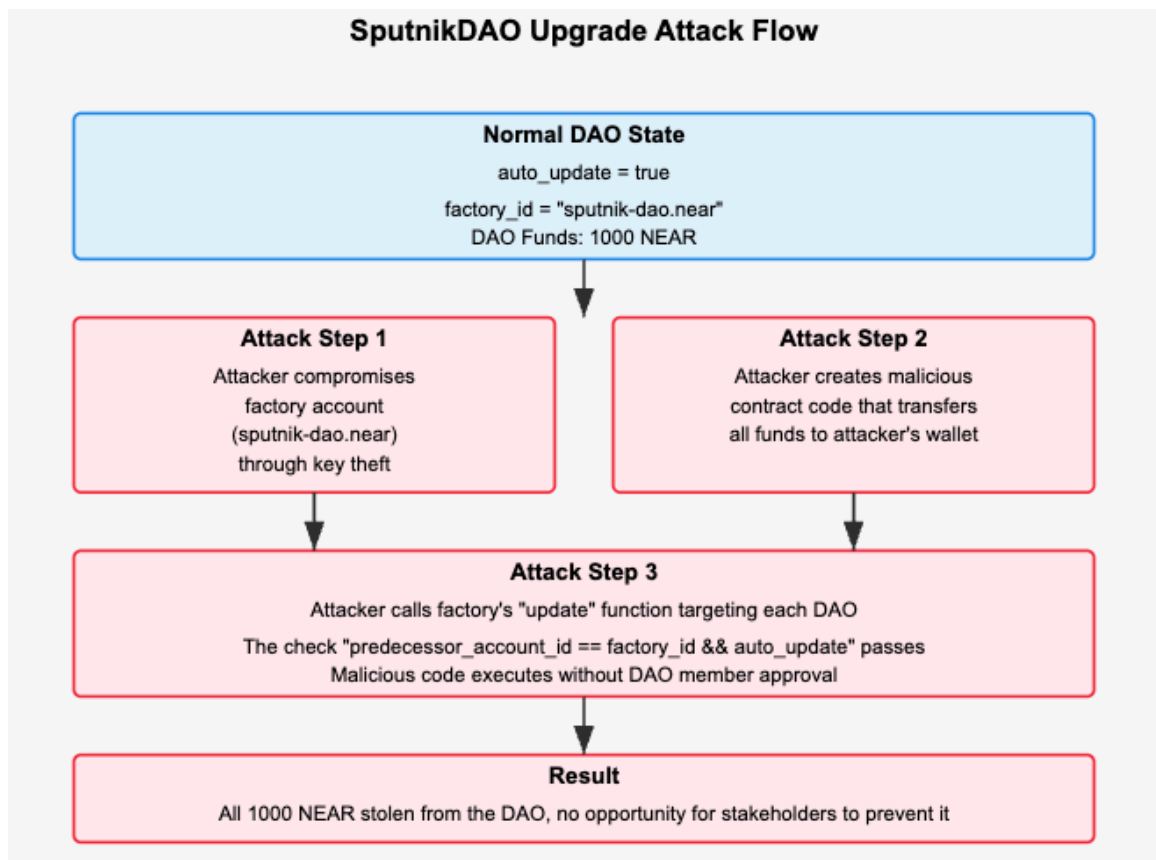
Status: **Fixed** -

<https://github.com/near-daos/sputnik-dao-contract/pull/233>

Description

The contract upgrade mechanism doesn't have adequate checks in place to ensure that the new contract code preserves funds. A compromised factory could push a malicious upgrade that potentially siphons funds.

The identified attack flow is shown below:



The applicable source code is shown below:

```
// upgrade.rs
pub fn update() {
    // Limited checks for who can upgrade
    let factory_info = internal_get_factory_info();
    let current_id = env::current_account_id();
    assert!(
        env::predecessor_account_id() == current_id
        || (env::predecessor_account_id() == factory_info.factory_id
            && factory_info.auto_update),
        "[]",
        ERR_MUST_BE_SELF_OR_FACTORY
    );
    // No verification of the new contract's integrity
}
```





Still Could Affect DAO and Users

The developer indicated that the “auto_update” flag is provided for those that do not wish to assume the risk of an automatically applied upgrade. This design still presents security concerns that should be addressed.

- **Single Point of Failure:** The factory account represents a single point of compromise that could affect multiple DAOs simultaneously
- **Immediate Execution:** Upgrades are applied immediately without any time/block delay period for review

Recommendation

The mitigating recommendation is implementing additional security such as Multi-Signature Factory Control and/or Time/Block Delayed Upgrades.



Staking Contract Immutability

H

Status: **Acknowledged**

Description

Once a staking contract has been set, it cannot be changed. If the staking contract becomes outdated or contains a critical bug, the DAO is forced to migrate to an entirely new contract, losing continuity.

The applicable source code is shown below:

```
// proposals.rs
ProposalKind::SetStakingContract { .. } => assert!(
    self.staking_id.is_none(),
    "ERR_STAKING_CONTRACT_CANT_CHANGE"
),
```

If the staking contract is compromised, there is no recovery/update mechanism. This immutability presents several risks:

1. **Security Vulnerabilities:** If security vulnerabilities are discovered in the staking contract, the DAO cannot upgrade to a patched version.
2. **Protocol Evolution:** As the NEAR ecosystem evolves, the DAO cannot adapt to new staking standards or protocols.
3. **Complete DAO Migration:** If the staking contract becomes problematic, the entire DAO must be recreated, requiring migration of all assets, proposals, and governance structures.
4. **Locked Tokens:** In extreme cases, tokens delegated through the staking contract could become permanently locked if critical bugs render the staking contract inoperable.

Recommendation

It is recommended to implement staking contract upgradeability using a high-threshold governance proposal, or a staking contract migration mechanism. If staking contracts must be immutable due to design constraints, then prominent warning should be added to the documentation regarding the permanence of the decision.

Note: Line 571 of proposals.rs indicates "// TODO: add more verifications." for this specific finding.



Missing Overflow/Underflow Checks

M

Status: **Acknowledged**

Description

`saturating_*` math operations are used, which can silently cap at boundaries rather than failing. In Rust smart contracts, `checked_*` and `saturating_*` functions provide different ways to handle integer overflows and underflows. `checked_*` functions return `Option<T>`. If an overflow/underflow occurs, they return `None`, allowing the program to handle the error explicitly. `saturating_*` functions return the value that wraps around to the nearest valid value (e.g., `u8::MAX + 1` becomes `u8::MAX`), preventing panics but potentially hiding errors. Exploitation would require specific conditions such as an extreme `locked_amount` value, which is not very likely.

The applicable source code is shown below:

```
// views.rs
pub fn get_available_amount(&self) -> NearToken {
    env::account_balance()
        .saturating_sub(self.get_locked_storage_amount())
        .saturating_sub(self.locked_amount)
}
```

Recommendation

It is recommended to replace `saturating_*` with `checked_*` to panic on overflow/underflow, ensuring errors are caught early.

References:

1. <https://medium.com/@dehvcurtis/top-rust-smart-contract-vulnerabilities-a-deep-dive-with-examples-eb36a84c800b>
2. <https://github.com/slowmist/solana-smart-contract-security-best-practices?tab=readme-ov-file#inaccurate-calculation-results>



Improper Validation on Token Transfers

M

Status: **Acknowledged**

Description

The **Transfer** proposal kind doesn't fully validate the token ID. An attacker could craft a transfer proposal with a malicious token contract that implements a non-standard token interface. Without proper validation, funds could be transferred to inaccessible addresses or lost due to interface mismatches.

The applicable source code is shown below:

```
// proposals.rs
ProposalKind::Transfer { token_id, msg, .. } => {
    assert!(
        !(token_id == OLD_BASE_TOKEN) || msg.is_none(),
        "ERR_BASE_TOKEN_NO_MSG"
    );
}
```

Recommendation

It is recommended to add validation for input parameters such as validating the **token_id** by checking if it implements NEP-141 (e.g., call `ft_metadata` to verify).



Insufficient Access Controls – delegate/undelegate

Status: **Acknowledged**

Description

The `delegate()` and `undelegate()` functions in `delegation.rs` only check if the caller is the staking contract, but there's no verification that the account being delegated to/from has given permission. A malicious staking contract could call `delegate()` with arbitrary account IDs, artificially inflating voting power for specific accounts.

The applicable source code is shown below:

```
pub fn delegate(&mut self, account_id: &AccountId, amount: U128) -> (U128, U128, U128) {
    let staking_id = self.staking_id.clone().expect("ERR_NO_STAKING");
    assert_eq!(
        env::predecessor_account_id(),
        staking_id,
        "ERR_INVALID_CALLER"
    );
    // Is any additional delegation authorization needed here?
}
```

The attack requires a malicious staking contract, which depends on the DAO's process for selecting "staking_id". If the DAO mistakenly sets a malicious contract, this vulnerability could be exploited.

Recommendation

Adding Authorization Tracking and/or Requiring Signature Verification.



Time-based Vulnerabilities

Status: **Acknowledged**

Description

The contract uses block timestamps for time-sensitive operations, which can be manipulated by validators within certain bounds. Validators could slightly manipulate timestamps to push proposals just over/under expiration thresholds. Timestamp-dependent logic could be vulnerable to frontrunning, where attackers observe pending transactions and submit their own with specific timing.

The applicable source code is shown below:

```
// policy.rs
pub fn proposal_status(
    &self,
    proposal: &Proposal,
    roles: Vec<String>,
    total_supply: Balance,
) -> ProposalStatus {
    if proposal.submission_time.0 + self.proposal_period.0 < env::block_timestamp() {
        // Proposal expired.
        return ProposalStatus::Expired;
    };
    // ...
}
```

Recommendation

It is recommended to use block height and/or time buffers for critical operations. For DAO contracts with proposal periods measured in days, the timestamp manipulation risk is generally low. A validator could manipulate the timestamps by a few seconds or minutes, but this would rarely change the outcome of a proposal that runs for multiple days.

If simplicity and human-readability are priorities:

- Continue using timestamps
- Consider adding a small buffer (e.g., 5 minutes) for critical timing decisions

If maximum determinism is the priority:

- Switch to block height
- Calculate approximate block height equivalents for your time periods
- Document the assumptions about block time for future reference



Unbounded Iterations

Status: **Acknowledged**

Description

Some functions iterate over collections without bound checks. An attacker could create many claims or members, causing iteration to consume excessive gas or potentially denying service. A maliciously crafted proposal with many votes or a policy with many roles could cause function calls to fail due to gas limits. Users attempting to interact with the contract may experience seemingly random failures when collections grow beyond a certain size.

The applicable source code is shown below:

```
// bounties.rs
fn internal_find_claim(&self, bounty_id: u64, claims: &[BountyClaim]) ->
Option<usize> {
    for i in 0..claims.len() {
        if claims[i].bounty_id == bounty_id {
            return Some(i);
        }
    }
    None
}
```

There are also other identified unbounded iterations:

1. In `Policy.proposal_status()` - Iterates through all roles
2. In `Policy.get_user_roles()` - Iterates through all roles
3. In `Policy.can_execute_action()` - Iterates through all permissions
4. In the `act_proposal` function - Iterates through votes

Recommendation

It is recommended to implement limits (`MAX_CLAIMS_PER_USER`) on iterations and collection sizes. For cases where iterations are unavoidable, it is recommended to make contract logic gas aware and to check if the gas limit is being approached.

Example:

```
// Constants defining maximum collection sizes

const MAX_CLAIMS_PER_USER: usize = 20;

const MAX_ROLES_IN_POLICY: usize = 10;

const MAX_PERMISSIONS_PER_ROLE: usize = 50;
```



Lack of Duplicate Proposal Prevention

Status: **Acknowledged**

Description

Proposals.rs does not prevent creating multiple proposals with identical parameters, which could lead to confusion, vote splitting, or potential denial-of-service attacks.

The applicable source code is shown below:

```
// proposals.rs
pub fn add_proposal(&mut self, proposal: ProposalInput) -> u64 {
    // Validate proposal...
    // Check permissions...

    // No check for duplicate proposals
    let id = self.last_proposal_id;
    // ...
    self.last_proposal_id += 1;
    // ...
}
```

Without duplicate detection, users could intentionally or accidentally:

1. Create multiple identical proposals, splitting the voting power across them
2. Resubmit failed proposals repeatedly without changes
3. Overwhelm the DAO with duplicate proposals as a form of spam
4. Create nearly identical proposals with subtle differences, causing voter confusion
5. Examples of spam governance on Osmosis linking to wallet drainers <https://www.mintscan.io/osmosis/proposals/895> & <https://www.mintscan.io/osmosis/proposals/894>

Recommendation

It is recommended to add duplicate detection by comparing proposal kinds and parameters. A cooldown period may be needed for resubmitting similar proposals after rejection and/or a unique constraint based on hash of the proposal parameters could be implemented.



Inconsistent Error Handling



Status: **Acknowledged**

Description

The contract uses a mix of `assert!()`, `expect()` and custom error messages:

The applicable source code is shown below:

```
// views.rs
pub fn get_proposal(&self, id: u64) -> ProposalOutput {
    let proposal =
self.proposals.get(&id).expect("ERR_NO_PROPOSAL");
    // ...
}
```

Recommendation

While this may not be a security issue, it is recommended to review each error handling method to determine if the proper method is being used, there are proper consistent prefixes, and relevant values to error messages are returned (for example: ID, account names, etc). While error handling isn't a direct security vulnerability, it can impact contact usability, maintenance, and integration. While this does not directly affect any funds, it's imperative for long term maintenance and user experience.

Example:

```
pub fn get_proposal(&self, id: u64) -> Result<ProposalOutput,
DaoError> {
    let proposal =
self.proposals.get(&id).ok_or(DaoError::ProposalNotFound { id
    })?;
```